Laporan Modul 2
Pemrograman Robotika dengan Kit iMCLab
Menggunakan Python



Oleh :

Muhammad Nur Firdaus (24081010263)

Universitas Pembangunan Nasional "Veteran" Jawa Timur

Fakultas Ilmu Komputer

Program Studi Informatika

2026

Daftar Isi

Daftar Isi 2
BAB I PENDAHULUAN 3
1.1 Latar Belakang 3
1.2 Rumusan Masalah 3
1.3 Tujuan Penelitian 3
BAB II LANDASAN TEORI 5
2.1 Sistem Pengendalian Motor Menggunakan Python dan ESP32 5
2.2 Komunikasi Serial dan Pustaka pySerial 5
2.3 Kendali Motor dengan Perintah Serial dan PWM 5
BAB III METODOLOGI 7
3.1 Metode praktikum 7
3.2 Alat dan Bahan 7
3.3 Program yang Digunakan 7
3.3.1 imclab_naik_turun.ino. 7
3.3.2 Tes_IMCLab_naik_turun.ipynb 7
3.4 Prosedur skenario 8
3.4.1 Skenario 1: Pengujian Koneksi dan Kontrol ON/OFF melalui Python 8
3.4.2 Skenario 2: Pengujian Variasi Nilai PWM melalui Python 8
3.4.3 Skenario 3: Pengujian Kenaikan dan Penurunan PWM secara Bertahap melalui Python 9
3.5 Analisis Hasil 9
BAB IV HASIL DAN PEMBAHASAN 10
4.1 Hasil Skenario 1: Pengujian Koneksi dan Kontrol ON/OFF melalui Python 10
4.2 Hasil Skenario 2: Pengujian Variasi Nilai PWM melalu Python 11
4.3 Hasil Skenario 3: Kenaikan dan Penurunan PWM Bertahap melalui Python 12
4.4 Pembahasan 13
KESIMPULAN 14

BAB I
PENDAHULUAN

1.1 Latar Belakang

Pengendalian motor jadi kunci di bidang robotika karena aktuator nggak akan bisa bekerja kalau tidak diatur dengan tepat. Di Modul 1 basic iMCLab, pengendalian motor DC pada kit iMCLab masih pakai Arduino IDE. Sekarang di Modul 2, pengendaliannya beralih pakai Python. Dengan pustaka pySerial, program Python bisa langsung membuka port serial, mengirim data, dan menerima balasan dari perangkat yang terhubung. Jadi, proses pengendaliannya nggak lagi lewat serial monitor.

Selama praktikum Modul 2, program "Tes_IMCLab_naik_turun.ipynb" dipakai untuk mengendalikan motor pada kit iMCLab lewat koneksi serial. Program ini punya fitur untuk mendeteksi port serial otomatis, membuka koneksi dengan ESP32, dan mengirim berbagai perintah, mulai dari ON/OFF, menaikkan atau menurunkan, hingga mengatur nilai PWM secara langsung. Python berfungsi sebagai penghubung utama lewat kabel USB, sementara semua pemrosesan perintah dan pengaplikasian sinyal PWM tetap dikerjakan oleh ESP32. Jadi, fokus praktikumnya sekarang terletak pada pemanfaatan Python sebagai antarmuka kendali dan observasi langsung bagaimana motor merespon setiap perintah yang dikirim.

1.2 Rumusan Masalah

1. Bagaimana cara menjalankan program Python untuk berkomunikasi dengan ESP32 pada kit iMCLab?
2. Bagaimana respons motor terhadap variasi nilai PWM yang dikirimkan melalui Python?
3. Bagaimana perubahan respons motor ketika nilai PWM diubah saat motor sedang berputar?
4. Bagaimana respons motor ketika nilai PWM dinaikkan dan diturunkan secara bertahap melalui perintah Python?

1.3 Tujuan Penelitian

1. Mengetahui cara menjalankan code python dan menghubungkannya dengan ESP32 melalui komunikasi serial.
2. Mengamati respons motor terhadap beberapa nilai PWM yang dikirimkan melalui Python.
3. Mengamati perubahan respons motor ketika nilai PWM diubah saat motor sedang berputar.

4. Memahami cara menaikkan dan menurunkan nilai PWM secara bertahap melalui perintah Python.

BAB II
LANDASAN TEORI

2.1 Sistem Pengendalian Motor Menggunakan Python dan ESP32

Sistem pengendalian motor pada praktikum ini melibatkan komputer, program Python, komunikasi serial, mikrokontroler ESP32, driver motor, dan motor DC. Setiap komponen memiliki fungsi yang berbeda, tetapi saling berhubungan dalam proses pengendalian. Komputer menjalankan program Python untuk menentukan perintah yang akan dikirimkan. Perintah tersebut dikirimkan melalui koneksi serial USB dan diterima oleh ESP32. Selanjutnya, ESP32 memproses perintah berdasarkan program yang telah diunggah dan menerapkan keluaran kendali melalui driver motor.

Dalam sistem ini, Python berfungsi sebagai antarmuka kendali, bukan sebagai pengendali PWM secara langsung. Pembangkitan sinyal PWM dan pengaturan keluaran motor tetap dilakukan oleh ESP32 melalui program imclab_naik_turun.ino. Program Python tersebut dapat dibuka dan dijalankan lewat Visual Studio Code ataupun Jupyter Notebook. Dengan demikian, perbedaan utama dengan Modul 1 terletak pada cara pengguna mengirimkan perintah, sedangkan program pengendali pada ESP32 tetap digunakan.

2.2 Komunikasi Serial dan Pustaka pySerial

Komunikasi serial intinya adalah pertukaran data satu per satu antara dua perangkat. Di praktikum ini, komunikasi serial menghubungkan komputer dengan ESP32 lewat USB. Kita menggunakan pySerial, pustaka Python yang mempermudah pembukaan port serial, pengaturan parameter komunikasi, pengiriman dan penerimaan data. Fungsi utamanya, seperti serial.Serial() untuk membuka koneksi dan write() buat mengirim data ke ESP32.

Pada notebook, baud rate dikonfigurasi ke 115200. Perintah dari Python dikirim sebagai teks yang diubah ke byte dan diakhiri karakter newline. Karakter newline ini jadi penanda bagi ESP32 bahwa satu perintah sudah selesai diterima. Setelah perintah diterima, ESP32 membalas lewat serial juga. Program Python membaca dan menampilkan balasan ini. Dengan begitu, pengguna bisa memantau perintah apa saja yang sudah dikirim dan respons yang muncul dari ESP32.

2.3 Kendali Motor dengan Perintah Serial dan PWM

Sistem pengendalian motor di Modul 2 tetap menggunakan program ESP32 imclab_naik_turun.ino seperti di Modul 1. Program ini menerima perintah ON/OFF dan pengaturan PWM melalui serial. Perintah yang diproses oleh ESP32 meliputi: karakter 1 untuk menyalakan motor, 0 untuk mematikan motor, + untuk menaikkan duty cycle sebesar 10, - untuk

menurunkan duty cycle sebesar 10, serta PWM:xxx untuk mengubah duty cycle ke nilai tertentu antara 0–255.

Pada Python, mode interaktif menerima perintah 1, 0, +, -, p<angka>, dan q. Misal, ketika mengetik p150, Python akan menerjemahkannya menjadi PWM:150, lalu mengirim ke ESP32 lewat serial. Huruf q digunakan untuk keluar dari mode interaktif sekaligus menutup koneksi serial. PWM (Pulse Width Modulation) sendiri adalah teknik mengatur proporsi waktu sinyal aktif dan tidak aktif dalam satu periode tertentu. Di praktikum, resolusinya 8 bit, jadi rentangnya 0–255. Semakin tinggi nilainya, semakin lama sinyal aktif dalam satu periode, tapi perubahan nilai PWM tidak otomatis berarti perubahan RPM tanpa diukur dengan metode yang tepat. Dasar teori motor DC dan PWM sudah dibahas di Modul 1. Pada Modul 2, fokusnya adalah bagaimana Python mengirim perintah ke ESP32, dan bagaimana reaksi motor bisa diamati setelah perintah diproses.

BAB III
METODOLOGI

3.1 Metode praktikum

Praktikum Modul 2 menggunakan metode eksperimen langsung dengan mengendalikan motor pada kit iMCLab melalui program Python. Pengujian dilakukan dengan menjalankan notebook Tes_IMCLab_naik_turun.ipynb pada komputer yang terhubung ke ESP32 melalui kabel USB. Program Python digunakan untuk mendeteksi port serial, membuka koneksi dengan ESP32, mengirimkan perintah pengendalian motor, dan menampilkan balasan dari perangkat. Program pada ESP32 tetap bertugas memproses perintah dan menerapkan sinyal kendali motor.

Pengujian meliputi pemeriksaan koneksi dan fungsi ON/OFF, pengujian beberapa nilai PWM, perubahan nilai PWM saat motor sedang berputar, serta kenaikan dan penurunan PWM secara bertahap. Hasil pengujian diamati melalui pesan yang ditampilkan pada Jupyter Notebook dan respons fisik motor.

3.2 Alat dan Bahan

1. Perangkat iMCLab yang dilengkapi dengan ESP32 dan motor DC.
2. Driver motor yang terpasang pada perangkat iMCLab.
3. Komputer atau laptop.
4. Kabel USB untuk menghubungkan ESP32 dengan komputer.
5. Arduino IDE untuk mengunggah program ke ESP32.
6. Python dan Jupyter Notebook untuk menjalankan program pengendalian.

3.3 Program yang Digunakan

3.3.1 imclab_naik_turun.ino.

Baud rate 115200, PWM 30.000 Hz dengan resolusi 8 bit (nilai 0–255). Pin motor: GPIO 27 dan 26 (arah, dikunci pada satu arah putar) dan GPIO 12 (PWM/enable). GPIO 2 menyalakan LED indikator saat duty cycle lebih dari 0. Konfigurasi ini mengikuti kode praktikum dan tidak berlaku umum untuk semua ESP32.

3.3.2 Tes_IMCLab_naik_turun.ipynb

Notebook Tes_IMCLab_naik_turun.ipynb dipakai untuk mengendalikan motor lewat Python. Di situ, program mengimpor pustaka serial, serial.tools.list_ports, time, dan sys. Program melakukan berbagai hal. Pertama, program menampilkan port serial yang tersedia, lalu mendeteksi port ESP32 otomatis, membuka koneksi serial, mengirim perintah, dan membaca respons dari

ESP32. Semua komunikasi berlangsung di baud rate 115200, dengan setiap perintah diakhiri karakter baris baru.

Begitu koneksi terbuka, notebook langsung menjalankan demonstrasi otomatis. Motor mulai dengan duty cycle 200. Duty cycle itu dinaikkan tiga kali, jadi 210, 220, dan 230, lalu diturunkan dua kali ke 220 dan 210. Setelah itu, PWM diatur ke 150 dan dilanjutkan dengan perintah OFF.Setelah urutan otomatis berakhir, pengguna masuk ke mode interaktif. Di sini, perintah seperti 1, 0, +, -, p<angka>, dan q bisa dimasukkan. Mode ini memungkinkan Anda melakukan pengujian lain tanpa harus mengulang semua sel kode. Ketika pengguna keluar, notebook akan mencoba mengirim perintah OFF dan menutup koneksi serial.

3.4 Prosedur skenario

3.4.1 Skenario 1: Pengujian Koneksi dan Kontrol ON/OFF melalui Python

1. Pastikan program imclab_naik_turun.ino sudah diunggah ke ESP32.
2. Hubungkan kit iMCLab ke komputer menggunakan kabel USB.
3. Tutup Serial Monitor Arduino IDE agar port serial dapat digunakan oleh program Python.
4. Buka Terminal atau Command Prompt, lalu jalankan perintah jupyter lab untuk membuka JupyterLab melalui browser.
5. Buka notebook Tes_IMCLab_naik_turun.ipynb.
6. Jalankan sel kode utama dengan tombol Run atau kombinasi Shift + Enter.
7. Amati daftar port serial, port ESP32 yang dipilih otomatis, dan pesan koneksi berhasil.
8. Amati urutan demonstrasi otomatis: motor ON (duty 200), kenaikan duty tiga kali (210, 220, 230), penurunan duty dua kali (220, 210), PWM 150, lalu motor OFF (duty 0).
9. Setelah mode interaktif muncul, masukkan perintah 1 dan amati apakah motor menyala.
10. Masukkan perintah 0 dan amati apakah motor berhenti.
11. Catat pesan dari Python (“>>”) dan balasan ESP32 (“ESP32:”) serta kondisi motor pada setiap perintah.

3.4.2 Skenario 2: Pengujian Variasi Nilai PWM melalui Python

1. Pastikan mode interaktif pada notebook siap menerima perintah dan motor dalam kondisi berhenti.
2. Masukkan perintah p160. Python menerjemahkannya menjadi PWM:160 dan mengirimkannya ke ESP32.

3. Amati kondisi motor serta pesan Python dan balasan ESP32.
4. Masukkan perintah 0 untuk menghentikan motor.
5. Masukkan perintah p200 (diterjemahkan menjadi PWM:200), lalu amati kondisi motor dan pesan yang tampil.
6. Masukkan perintah 0 untuk menghentikan motor.
7. Catat hasil pengamatan untuk kedua nilai PWM.

3.4.3 Skenario 3: Pengujian Kenaikan dan Penurunan PWM secara Bertahap melalui Python

1. Pastikan notebook masih terhubung dengan ESP32 dan mode interaktif siap digunakan.
2. Masukkan perintah 1 untuk menyalakan motor. Berdasarkan program, motor mulai dengan duty cycle 200 apabila nilai sebelumnya 0.
3. Masukkan perintah + sebanyak dua kali dan amati perubahan duty cycle menjadi 210 lalu 220.
4. Masukkan perintah - sebanyak empat kali berurutan dan amati penurunan duty cycle menjadi 210, 200, 190, dan 180.
5. Catat pesan Python, balasan ESP32, dan respons motor pada setiap perintah.
6. Masukkan perintah 0 untuk menghentikan motor setelah pengujian selesai.

3.5 Analisis Hasil

Analisis data dilakukan secara deskriptif dengan membandingkan perintah yang diberikan melalui Python, nilai PWM yang diterapkan, respons motor yang diamatianalisis difokuskan pada keberhasilan deteksi port, pembukaan koneksi serial, serta pelaksanaan demonstrasi otomatis. Pada skenario pertama, analisis difokuskan pada kesesuaian antara perintah ON/OFF dan kondisi motor. Pada skenario kedua, respons motor dibandingkan berdasarkan beberapa nilai PWM. Pada skenario ketiga, analisis diarahkan pada perubahan respons motor ketika nilai PWM diubah saat motor sedang berputar. Pada skenario keempat, analisis difokuskan pada perubahan respons motor ketika nilai PWM dinaikkan dan diturunkan secara bertahap.

BAB IV
HASIL DAN PEMBAHASAN

4.1 Hasil Skenario 1: Pengujian Koneksi dan Kontrol ON/OFF melalui Python

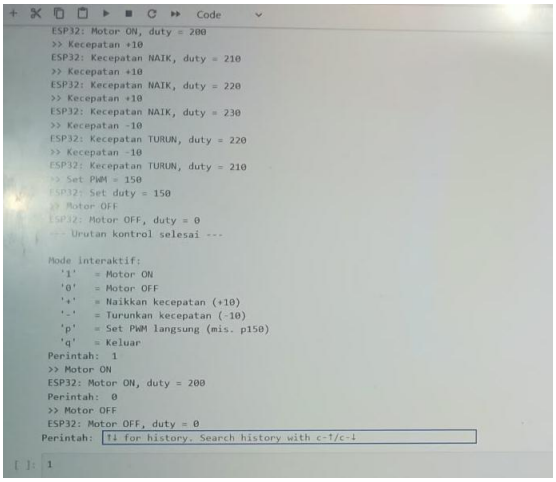
Pengujian memakai imclab_naik_turun.ino pada ESP32 dan notebook Tes_IMCLab_naik_turun.ipynb pada komputer. Setelah sel kode dijalankan, program otomatis menjalankan urutan demonstrasi. Setiap perintah dari Python (ditandai ">>") dibalas ESP32 (ditandai "ESP32:"), sehingga koneksi serial antara Python dan ESP32 terbukti berfungsi.

Tabel 4.1 Urutan demonstrasi otomatis

Langkah	Pesan Python	Balasan ESP32
Motor ON	>> Motor ON	Motor ON, duty = 200
Naik (1)	>> Kecepatan +10	Kecepatan NAIK, duty = 210
Naik (2)	>> Kecepatan +10	Kecepatan NAIK, duty = 220
Naik (3)	>> Kecepatan +10	Kecepatan NAIK, duty = 230
Turun (1)	>> Kecepatan -10	Kecepatan TURUN, duty = 220
Turun (2)	>> Kecepatan -10	Kecepatan TURUN, duty = 210
Set PWM	>> Set PWM = 150	Set duty = 150

Setelah demonstrasi selesai, program masuk ke mode interaktif. Perintah 1 dimasukkan dan ESP32 membalas "Motor ON, duty = 200" sehingga motor menyala. Perintah 0 (OFF) dimasukkan dan ESP32 membalas "Motor OFF, duty = 0" sehingga motor berhenti

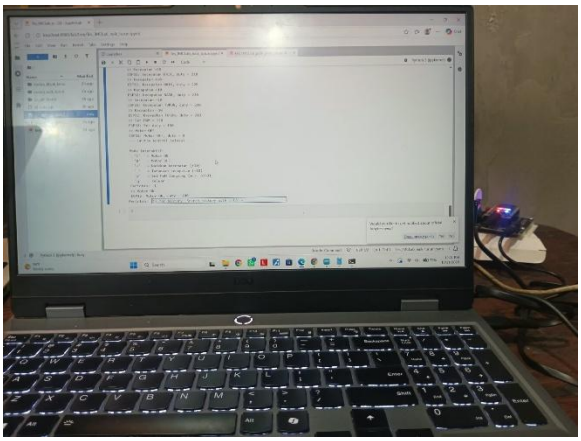
Dokumentasi gambar:



Gambar 4.1 Demonstrasi Otomatis

Tabel 4.2 Tabel Keterangan ON/OFF

Perintah	Pesan Python	Balasan ESP32
1	>> Motor ON	Motor ON, duty = 200
0	>> Motor OFF	Motor OFF, duty = 0



```
Perintah:  1
>> Motor ON
ESP32: Motor ON, duty = 200
Perintah:  0
>> Motor OFF
ESP32: Motor OFF, duty = 0
Perintah:  ↑↓ for history. Search history with c-↑/c-↓
```

Gambar 4.2 Kondisi ON

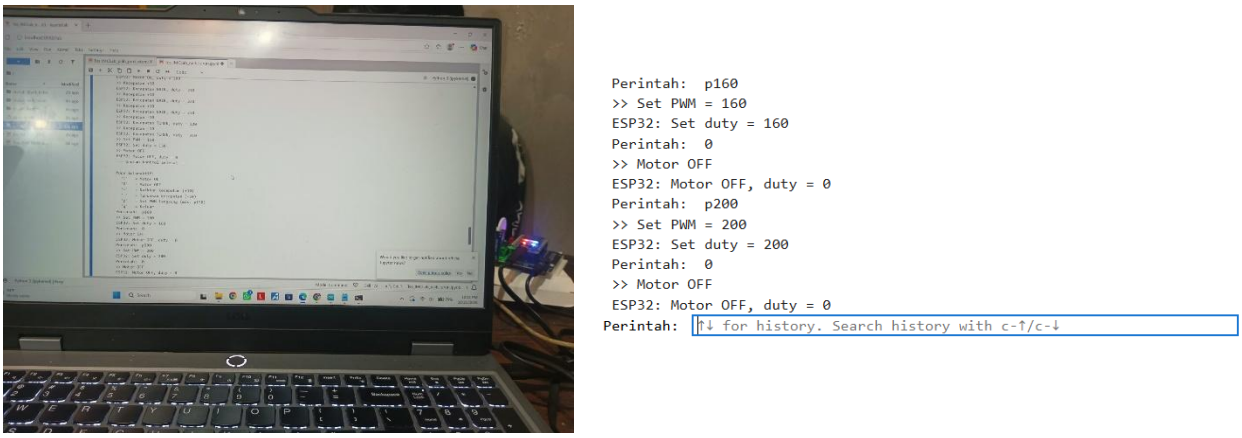
4.2 Hasil Skenario 2: Pengujian Variasi Nilai PWM melalu Python

Pada mode interaktif, perintah p160 dimasukkan lalu diterjemahkan Python menjadi PWM:160 dan dikirim ke ESP32. Motor dihentikan dengan perintah 0, kemudian pengujian diulang dengan p200. Motor menyala pada kedua nilai PWM dan berhenti setelah perintah 0.

Tabel 4.3 Variasi nilai PWM

Perintah	Pesan Python	Balasan ESP32	Kondisi motor
p160	>> Set PWM = 160	Set duty = 160	Motor menyala
0	>> Motor OFF	Motor OFF, duty = 0	Motor mati
p200	>> Set PWM = 200	Set duty = 200	Motor menyala
0	>> Motor OFF	Motor OFF, duty = 0	Motor mati

Dokumentasi gambar:



Gambar 4.3 Uji Skenario 2

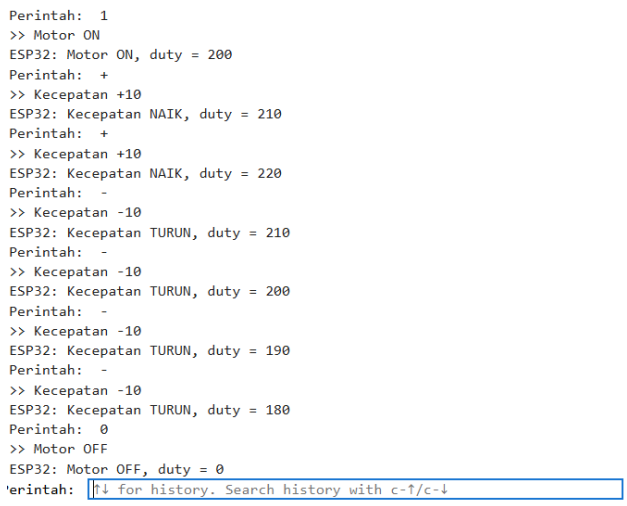
4.3 Hasil Skenario 3: Kenaikan dan Penurunan PWM Bertahap melalui Python

Motor dinyalakan dengan perintah 1 pada duty 200. Perintah + dikirim dua kali sehingga duty naik menjadi 210 lalu 220 dan motor bertambah cepat. Perintah - dikirim empat kali sehingga duty turun menjadi 210, 200, 190, dan 180 dan motor bertambah lambat. Pengujian diakhiri dengan perintah 0 (OFF) dan motor mati. Setiap + menaikkan duty sebesar 10 dan setiap - menurunkannya sebesar 10.

Tabel 4.4 Kenaikan dan penurunan PWM bertahap

Perintah	Pesan Python	Balasan ESP32
1	>> Motor ON	Motor ON, duty = 200
+	>> Kecepatan +10	Kecepatan NAIK, duty = 210
+	>> Kecepatan +10	Kecepatan NAIK, duty = 220
-	>> Kecepatan -10	Kecepatan TURUN, duty = 210
-	>> Kecepatan -10	Kecepatan TURUN, duty = 200
-	>> Kecepatan -10	Kecepatan TURUN, duty = 190
-	>> Kecepatan -10	Kecepatan TURUN, duty = 180
0	>> Motor OFF	Motor OFF, duty = 0

Dokumentasi gambar:



Gambar 4.4 Uji Skenario 3

4.4 Pembahasan

Pada Skenario 1, Python bisa berkomunikasi lancar dengan ESP32: setiap kali mengirim perintah, ESP32 langsung membalas, dan hasilnya muncul di notebook. Kalau kirim perintah 1, motor menyala. Kalau kirim 0, motor mati. Di Skenario 2, motor bisa langsung jalan pada nilai PWM tertentu dengan perintah p<angka> tanpa harus menyalakan dulu lewat perintah 1. Sementara di Skenario 3, perintah + bikin motor tambah cepat dan perintah - bikin motor melambat, dengan duty cycle bergeser 10 setiap kali, sesuai respons ESP32.

Dibandingkan dengan Modul 1, kodenya masih sama (imclab_naik_turun.ino) dan hasilnya juga mirip: motor bisa menyala, mati, dan kecepatannya bisa diatur naik atau turun lewat perintah. Bedanya, sumber perintah di Modul 1 pakai Serial Monitor dari Arduino IDE, sedangkan di Modul 2 pakai Python lewat pySerial. Di Python, perintah dikirim lengkap dengan karakter akhir baris secara otomatis. Hasil pengamatan tetap bersifat kualitatif. Program tidak menghitung RPM, jadi tidak ada pengukuran kecepatan motor. Akibatnya, kita nggak bisa menentukan hubungan numerik antara nilai PWM dan kecepatan motor.

KESIMPULAN

Dari praktikum Modul 2 Pemrograman Robotika memakai kit iMCLab dengan Python, terlihat motor DC bisa dikendalikan dengan program Python yang terhubung ke ESP32 lewat serial USB. Di Skenario 1, Python sukses buka koneksi ke ESP32. Saat mengirim perintah 1, motor langsung menyala, dan dengan perintah 0, motor mati. Skenario 2 berbeda, bisa mengatur motor langsung ke nilai PWM tertentu, misalnya, 160 atau 200. Lalu Skenario 3, bereksperimen mencoba naik dan turunkan PWM perlahan-lahan, naik atau turun 10 tiap perintah, dan motor merespons sesuai, makin cepat atau lambat.

Dari keseluruhan praktikum, jelas Python bisa jadi antarmuka kendali yang praktis menggantikan Serial Monitor, sedangkan ESP32 tetap yang tangani sinyal PWM-nya. Hasilnya masih konsisten dengan Modul 1. Satu hal, kecepatan motor dilakukan penilaian kualitatif karena program ini belum menangkap RPM dalam angka pasti.